Custom DLP Policy Development — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Custom DLP Policy Development (Microsoft Purview) 4
What you get . . . 4
What this delivers . . . 4
The deployment model . . . 4
Scope boundary . . . 5
Outcome . . . 5
1. Solution Overview — Custom DLP Policy Development (Microsoft Purview) 5
What this solution is . . . 5
Why this matters . . . 6
How it works . . . 6
Where custom DLP conditions come from . . . 7
What you can control . . . 7
Limitations (be honest with buyers) . . . 8
Naming convention (used in every doc and script) . . . 8
The standard deployment model . . . 8
2. Requirements Guide — Custom DLP Policy Development (Microsoft Purview) 9
2.1 DLP licensing for the workloads you target . . . 9
2.2 Least-privilege DLP administration role . . . 9
2.3 Security & Compliance PowerShell toolchain . . . 9
2.4 In-scope workloads (and endpoint / cloud-app onboarding, if targeted) . . . 9
2.5 The sensitive information types / classifiers the conditions need . . . 10
2.6 A documented data-protection intent (from stakeholders) . . . 10
2.7 A pilot scope, simulation mode, and change control . . . 10
Checklist . . . 10
Global Requirements — Microsoft Purview Solutions 11
1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 11
2. Purview roles (least-privilege) and portal access . . . 11
3. Security & Compliance PowerShell + Graph toolchain . . . 12
4. Data sources & scope in scope . . . 12
5. Compliance, records, privacy & legal stakeholders . . . 12
6. A pilot scope and change control . . . 12
3. Implementation Guide — Custom DLP Policy Development (Microsoft Purview) 13
Step 1 — Assess the DLP environment (Assess) . . . 13
Step 2 — Design conditions, actions, and exceptions (Design) . . . 14
Step 3 — Configure the policy in SIMULATION / test mode (Configure, simulate-first) . . . 14

Step 4 — Pilot: let the policy simulate (Pilot) . . . 15
Step 5 — Validate and tune against the match evidence (Validate) . . . 15
Step 6 — Expand to enforcement, staged, under change control (Expand) . . . 15
5. Testing Guide — Custom DLP Policy Development (Microsoft Purview) 16
Test 1 — Environment discovery is complete . . . 16
Test 2 — Policy creation dry-run makes no changes . . . 16
Test 3 — Policy is created in simulation / test mode, on the pilot scope . . . 17
Test 4 — Simulation generates matches with no user impact . . . 17
Test 5 — Known-sensitive samples are caught (false-negative check) . . . 17
Test 6 — Approved flows are not caught (false-positive / exception check) . . . 17
Test 7 — Tuning changes are previewed before applied . . . 17
Test 8 — Enforcement cut-over is staged and reversible . . . 17
Pass/fail gate . . . 18
6. Troubleshooting Guide — Custom DLP Policy Development (Microsoft Purview) 18
DLP does not appear, or a workload can't be targeted . . . 18
`Connect-IPPSSession` fails or DLP cmdlets are not found . . . 18
A referenced sensitive info type "does not exist" . . . 18
The policy is over-firing (too many false positives) . . . 18
The policy misses known-sensitive data (false negatives) . . . 19
A pilot user sees a policy tip or a block during simulation . . . 19
Simulation shows no matches at all . . . 19
Matches can't be exported by the reporting script . . . 19
Endpoint DLP policy has no effect on devices . . . 19
An approved business flow is being blocked after enforcement . . . 20
7. Best Practices — Custom DLP Policy Development (Microsoft Purview) 20
8. Deliverables Summary — Custom DLP Policy Development (Microsoft Purview) 21
Documentation . . . 21
PowerShell scripts . . . 22
What the buyer receives . . . 23
Outcome . . . 23
Appendix A — PowerShell Scripts 24

Executive Summary — Custom DLP Policy Development (Microsoft Purview)

Microsoft Purview ships with built-in, templated DLP policies — but they rarely fit real data. Your organisation has its own document types, its own identifiers, and its own approved sharing paths, and a template run unchanged produces noise: false positives that train users to click through every warning, and false negatives that miss the data you actually care about. Worse, a DLP block shipped straight to enforcement can stop legitimate business email and file sharing on day one. This solution fixes that: it develops **custom DLP policies in Microsoft Purview, tuned to your exact data**, piloted safely, and promoted to enforcement only once the evidence says they're ready.

It gives you a repeatable way to design **tailored conditions** — the right sensitive information types at the right confidence and instance counts, plus labels and content/recipient conditions — with deliberate **exceptions** for approved business flows, and **staged actions** (audit → notify → restrict/block → encrypt). Every policy is created in **simulation / test mode first**, so it observes and records matches with **zero user impact**; it is then tuned against that evidence and moved to enforcement under change control. All of it is done with the Microsoft Purview portal and Security & Compliance PowerShell — no third-party tooling.

This is a complete, deploy-it-today package for building and tuning custom DLP policies, written for a competent M365 / compliance administrator.

What you get

• A plain-language Solution Overview of how custom DLP maps to your data: conditions, actions, exceptions,

and the simulate-first tuning loop.

• A Requirements Guide with verify / implement-if-missing for every prerequisite, including which workloads

your licensing covers and which sensitive info types your conditions need.

• An Implementation Guide of precise, ordered portal + PowerShell steps, with the policy created in

simulation, each policy/rule named, `-WhatIf` previews, and pass/fail gates + rollback at every stage.

• Five PowerShell scripts: environment discovery, a policy author/tuner (`-WhatIf` by default, simulation

by default), a policy validator, a match reporter for tuning, and a scope/health helper.

• Testing and Troubleshooting guides for a confident, safe rollout.

What this delivers

Custom DLP policies that fit your data — tailored conditions and sensitive info types, exceptions that keep approved flows working, and staged actions — piloted in simulation with **no user impact**, tuned until the false-positive and false-negative rates hit the agreed target, and only then enforced. The result is data-loss protection that is precise, credible with users, and defensible in an audit.

The deployment model

Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce).

Scope boundary

This solution develops the **policies and rules**. The data classes they reference — **custom sensitive information types, Exact Data Match, document fingerprints, trainable classifiers, and sensitivity labels** — are prerequisites built in their own dedicated solutions and consumed here as conditions. **Endpoint** and **cloud-app** coverage have their own onboarding; this policy targets those locations once they're in place. **What data is sensitive, and which regulations apply, is deferred to the organisation's compliance, privacy, and legal functions** — this solution provides the technical configuration and the match evidence, not the determination.

Outcome

DLP tuned to your exact data in Microsoft Purview — custom policies with tailored conditions, sensitive info types, exceptions, and tuning — delivered, validated in simulation, and enforced defensibly.

1. Solution Overview — Custom DLP Policy Development (Microsoft Purview)

What this solution is

Microsoft Purview ships with a large set of **built-in / templated DLP policies** (financial, medical, privacy, regional PII, and more). They are a reasonable starting point, but they rarely fit *your* data: your organisation has its own document types, its own identifiers, its own approved sharing paths, and its own tolerance for friction. Run a template unchanged and you get noise — false positives that train users to click through warnings, and false negatives that miss the data you actually care about.

This solution builds **custom Data Loss Prevention policies in Microsoft Purview** that are tuned to your exact data. It delivers, end to end:

- **Custom DLP policies** authored against the workloads you choose — Exchange Online, SharePoint Online,

OneDrive, Teams, and endpoint devices — rather than a one-size template.

- **Tailored conditions and rules** — the right combination of **sensitive information types (SITs)**,

confidence levels, instance counts, sensitivity labels, and content/recipient conditions that match how your data really looks.

- **Deliberate exceptions** — the allowed business flows (approved recipients/domains, service mailboxes,

specific sites) encoded as rule exceptions so legitimate work is not blocked.

- **A tuning loop** — the policy is piloted in **simulation / test mode first**, matches are reviewed,

false positives and negatives are driven down, and only then is the policy moved to enforcement.

Everything is done with the **Microsoft Purview portal** (purview.microsoft.com) and **Security & Compliance PowerShell** (`Connect-IPPSSession`) — no third-party tooling. The five included scripts author, validate, and report on the policies **safe-by-default** (read-only, or `-WhatIf` until you pass `-Commit`).

> ***This is custom policy *development*, not the DLP platform build-out.*** *It assumes the Purview foundation and DLP prerequisites are in place (licensing, roles, the S&C PowerShell toolchain, the in-scope workloads, and — for endpoints — device onboarding). It focuses squarely on* ***designing, configuring, simulating, tuning, and enforcing custom DLP policies****. Where a prerequisite belongs to another solution (endpoint onboarding, custom sensitive info types, Exact Data Match, sensitivity labels), this guide points to it rather than duplicating it.*

Why this matters

• **Templates don't know your data.** A built-in "U.S. PII" policy fires on any SSN-shaped string anywhere; it doesn't know that your finance shared mailbox legitimately handles them, or that your real exposure is a bespoke customer-ID format the template has never heard of. Custom policies encode *your* reality.

• **False positives are a security risk, not just an annoyance.** Every over-firing rule trains users to dismiss DLP warnings — so the one that matters gets dismissed too. Tuning is what makes DLP credible.

• **Un-piloted DLP breaks work.** A block action shipped straight to enforcement can stop legitimate business email or file sharing on day one. Piloting in **simulation/test** first is the difference between a controlled rollout and an outage.

• **Defensibility.** A policy that was designed to a documented data-protection intent, piloted, tuned against evidence, and enforced under change control is one you can stand behind in an audit — not a guess.

How it works

1. **Assess.** Read the tenant read-only: existing DLP policies and rules, which sensitive info types are available, the in-scope workloads, and the DLP licensing. Establish what's already firing (or over- firing) before adding anything.

1. **Design.** Decide, per data type, the **conditions** (which SITs, confidence, instance count, labels, recipients), the **actions** (audit only, notify/policy tip, restrict access, block, encrypt), and the **exceptions** (approved flows) — mapped to the data-protection intent your stakeholders provide.

1. **Configure.** Author the custom policy and its rules **in simulation / test mode** — the policy is created with `TestWithoutNotifications` (or `TestWithNotifications`) so it evaluates content and records matches **without blocking anything or notifying users**. Changes are previewed with `-WhatIf` first.

1. **Pilot (simulate).** Let the policy run in simulation over the pilot scope, generating match data with no user impact. This is the mandatory high-impact-change gate for the Purview category.

1. **Validate & tune.** Review the simulation matches: drive down false positives (tighten conditions, add exceptions, raise confidence/instance thresholds) and false negatives (broaden or add SITs). Re-simulate until the match quality passes the gate.

1. **Expand (enforce).** Move the tuned policy to **enforcement** (`Enable`) — first with policy tips / notifications, then with the restrictive action — under change control, keeping the rollback path ready.

Where custom DLP conditions come from

A custom DLP rule is a combination of building blocks. The design step chooses from:

Building block	What it does	Example
Sensitive information type (SIT)	Pattern/keyword/checksum match for a data class	Built-in "Credit Card Number"; a **custom SIT** for your customer-ID format
Confidence level	How strict the SIT match must be (low/medium/high)	Require *high* confidence to cut false positives
Instance count (min/max)	How many matches trigger the rule	1–9 for a tip, 10+ for a block
Sensitivity label	Content carrying a specific label	Trigger on the "Highly Confidential" label
Content/recipient conditions	Where content is going / who to	Sent **outside the organisation**; to a non-approved domain
Exceptions (`ExceptIf…`)	Approved flows that must *not* trigger	Except when recipient is an approved partner domain
Actions	What happens on a match	Audit → policy tip/notify → restrict/block → encrypt

> ***Custom SITs, Exact Data Match, document fingerprinting, and sensitivity labels are inputs, not this solution's build.*** *If a condition needs a data class Purview doesn't have out of the box, that SIT is built in its own dedicated solution and then *consumed* here as a condition. This solution develops the **policies and rules**; it names the building blocks it uses.*

What you can control

• Which **workloads** the policy applies to (Exchange, SharePoint, OneDrive, Teams, endpoints) and its scope (all users vs a pilot group).

• The **conditions**: which SITs, at which confidence, at which instance counts, plus labels and content/recipient conditions.

• The **exceptions** that keep approved business flows working.

• The **actions** and their staging: audit-only → policy tip/notify → restrict/block → encrypt.

• The **mode**: simulation/test (no impact) → enforcement — and the pace at which you move between them.

Limitations (be honest with buyers)

• **DLP is licensing-dependent.** Baseline DLP for Exchange/SharePoint/OneDrive/Teams is broadly

available; **Endpoint DLP** and some advanced classification (trainable classifiers, EDM) require **Microsoft 365 E5** or the **E5 Compliance** add-on. Capability availability is **flagged "verify your licensing/tenant"**, never assumed. (Global Requirement 1.)

• **The portal is evolving.** Purview DLP portal paths and the simulation experience change over time.

Paths here are cited as of authoring and marked **"verify in your tenant."**

• **This solution builds policies, not the data classes underneath.** Custom sensitive info types, Exact

Data Match, document fingerprinting, and the sensitivity-label taxonomy are prerequisites developed in their own solutions and consumed here as conditions.

• **Endpoint and cloud-app coverage have their own onboarding.** Endpoint DLP needs devices onboarded to

Purview; cloud-app DLP needs Defender for Cloud Apps — those roll-outs are separate solutions; this one authors the policy that targets them once they're in place.

• **What data is sensitive, and which regulations apply, is not ours to decide.** The data-protection

intent, retention/records implications, and regulatory applicability are **deferred to compliance, privacy, and legal**; this solution provides the technical configuration and the match evidence.

Naming convention (used in every doc and script)

Artifacts this solution creates or captures follow `Custom DLP – <Purpose> – <Stage>` and are reused verbatim across the docs, scripts, and portal. The placeholder data type `Sensitive Data` stands in for your real class (e.g. `Financial PII`, `Customer ID`) — rename consistently.

Artifact	Name
Environment / existing-DLP discovery export	`Custom DLP – Discovery – Assess`
Policy design (conditions/actions/exceptions matrix)	`Custom DLP – Sensitive Data – Design`
The custom DLP policy (piloted in simulation)	`Custom DLP – Sensitive Data – Pilot`
The rule inside the policy	`Custom DLP Rule – Sensitive Data – R1`
Simulation validation snapshot	`Custom DLP – Simulation – Validate`
Match / incident report	`Custom DLP – Matches – Report`
Workload & SIT scope map	`Custom DLP – Scope – Map`

The standard deployment model

> *Assess the environment → design conditions/actions/exceptions → configure the policy in simulation/test → pilot (simulate, no user impact) → validate & tune against the match evidence → expand to enforcement under change control. Pattern:* ***Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce).***

2. Requirements Guide — Custom DLP Policy Development (Microsoft Purview)

Verify **every** requirement before authoring a policy. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-purview.md` (the six shared requirements: compliance licensing, least-privilege Purview roles, the S&C PowerShell + Graph toolchain, in-scope data sources, compliance/records/privacy/legal stakeholders, and a pilot scope with change control) — those are referenced here, not repeated.

2.1 DLP licensing for the workloads you target

Description: The **DLP licensing** the target workloads require — baseline DLP for Exchange Online, SharePoint Online, OneDrive, and Teams is broadly available; **Endpoint DLP** (devices) and advanced classification inputs (trainable classifiers, Exact Data Match) require **Microsoft 365 E5** or the **E5 Compliance** add-on (Global Requirement 1). **Why:** A policy that targets a workload/condition your plan doesn't license will not apply there. **Verify:** Microsoft 365 admin center → `Billing → Licenses`; run `DLPCUSTOM-Assess-DlpEnvironment.ps1`, which reports the DLP-relevant SKUs and whether Endpoint DLP is in scope. Confirm DLP appears in the Purview portal (`Data loss prevention`). **Implement:** Assign/purchase the required licensing before targeting an unlicensed workload; otherwise scope the policy to the workloads you can license and record the rest as deferred.

2.2 Least-privilege DLP administration role

Description: An account in a least-privilege role that can author DLP — **Compliance Administrator** or **Compliance Data Administrator**, or a **DLP Compliance Management** role group — rather than blanket Global Administrator (Global Requirement 2). **Why:** Purview's fine-grained role model limits exposure; DLP authoring does not require Global Admin. **Verify:** Purview portal → `Settings → Roles & scopes → Role groups` (verify path) — confirm membership; `Connect-IPPSSession` succeeds and `Get-DlpCompliancePolicy` returns without an authorization error. **Implement:** Add the account to the least-privilege DLP-capable role group under change control.

2.3 Security & Compliance PowerShell toolchain

Description: A workstation with the **ExchangeOnlineManagement** module (provides `Connect-IPPSSession` for Security & Compliance PowerShell), and a modern-auth / MFA admin account (Global Requirement 3). DLP policy cmdlets (`*-DlpCompliancePolicy`, `*-DlpComplianceRule`) live in S&C PowerShell. **Why:** Every script in this solution authors, validates, or reports on DLP through S&C PowerShell. **Verify:** `Get-Module -ListAvailable ExchangeOnlineManagement`; `Connect-IPPSSession` succeeds and `Get-DlpCompliancePolicy` runs. `DLPCUSTOM-Assess-DlpEnvironment.ps1` installs a missing module and reports connectivity. **Implement:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser`; confirm modern-auth sign-in.

2.4 In-scope workloads (and endpoint / cloud-app onboarding, if targeted)

Description: The **workloads in scope** for the policy — Exchange, SharePoint, OneDrive, Teams — and, **if** you target them, **endpoint devices onboarded to Purview** (for Endpoint DLP) and **Defender for Cloud Apps** connected (for cloud-app DLP). Global Requirement 4. **Why:** DLP applies per location; endpoints and cloud apps are **not** covered until onboarded/connected, which are separate roll-outs. **Verify:** Run `DLPCUSTOM-Scope-Health.ps1` — it confirms the M365 workloads and reports endpoint onboarding readiness. In the portal, `Settings → Device onboarding` shows onboarded devices (verify path). **Implement:** Target only the workloads that are ready. Endpoint onboarding and cloud-app connection are performed in their dedicated solutions; this policy targets those locations **once they exist**.

2.5 The sensitive information types / classifiers the conditions need

Description: The **sensitive information types** (built-in or **custom**), **Exact Data Match** schemas, **document fingerprints**, **trainable classifiers**, or **sensitivity labels** that your rule conditions will reference — available in the tenant *before* you author the policy. **Why:** A rule condition can only reference a SIT/label that exists; a custom identifier (e.g. your customer-ID format) must be built first, or the rule can't detect it. **Verify:** Run `DLPCUSTOM-Assess-DlpEnvironment.ps1` — it lists available SITs (`Get-DlpSensitiveInformationType`) and flags whether the ones your design needs are present. Sensitivity labels via `Get-Label` (verify). **Implement:** Build any missing custom SIT / EDM schema / fingerprint / trainable classifier in its own solution first, then consume it here as a condition. Do **not** invent a SIT inside this policy work.

2.6 A documented data-protection intent (from stakeholders)

Description: A written **data-protection intent** from the compliance/privacy/security stakeholders (Global Requirement 5): which data classes matter, what should happen on a match (audit / warn / block / encrypt), which flows are approved (exceptions), and where the regulatory line is. **Why:** DLP conditions and actions must encode enterprise policy, not technical defaults; **what is sensitive and which regulations apply are determinations for compliance/privacy/legal**, not the engineer. **Verify:** The intent is documented and approvers are named; each planned rule maps to a stated intent. **Implement:** Capture the intent (data classes, desired actions, approved flows, regulatory drivers) before authoring — record it as `Custom DLP – Sensitive Data – Design`. Escalate any determination.

2.7 A pilot scope, simulation mode, and change control

Description: A named **pilot scope** (users/sites/devices) to simulate on, confirmation that DLP **simulation / test mode** is available (`TestWithoutNotifications` / `TestWithNotifications`), and a **change-control** process for the move to enforcement (Global Requirement 6). **Why:** Every Purview policy is piloted before enforcement; DLP block/encrypt actions are high-impact and must be simulated first so they cannot break legitimate work. **Verify:** The pilot scope is defined; a new/updated policy can be created in a test mode; a change window/approval path exists for the enforcement cut-over. **Implement:** Define the pilot scope and agree change control before moving any policy out of simulation.

Checklist

- [ ] DLP licensing confirmed for every targeted workload (Endpoint DLP / advanced inputs need E5 / E5 Compliance)

- Least-privilege DLP admin role confirmed; Global Admin not required for authoring
- ExchangeOnlineManagement installed; `Connect-IPPSSession` + `Get-DlpCompliancePolicy` succeed
- In-scope workloads agreed; endpoints/cloud-apps onboarded **if** targeted (else deferred)
- Required SITs / labels / EDM / fingerprints exist in the tenant (or built first in their own solution)
- Data-protection intent documented and approvers named (`Custom DLP – Sensitive Data – Design`)
- Pilot scope defined; simulation/test mode available; change control agreed for enforcement

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Custom DLP Policy Development (Microsoft Purview)

Follow these steps **in order**. Authoring is done in the **Microsoft Purview portal** (`https://purview.microsoft.com` → **Data loss prevention**) and **Security & Compliance PowerShell** (`Connect-IPPSSession`). Policy creation/changes are previewed with `-WhatIf`, applied with `-Commit`, and the policy is **created in simulation / test mode first** — it only moves to enforcement after the match evidence passes the gate. Artifact names follow the `Custom DLP – <Purpose> – <Stage>` convention from the Overview and are reused verbatim; the placeholder data type `Sensitive Data` stands in for your real class.

> ***Model:*** *Assess the environment, design conditions/actions/exceptions, configure the policy in simulation, pilot (simulate — no user impact), validate & tune against the matches, then expand to enforcement under change control. Pattern:* ***Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce). Portal paths are cited as of authoring and the Purview portal is evolving — verify each path in your own tenant.*** *Capability availability is licensing-dependent — verify against your plan (Requirement 2.1).* ***Never*** *create a custom DLP policy directly in* `Enable` *mode: always simulate first.*

Step 1 — Assess the DLP environment (Assess)

1. Open the Purview portal → **Data loss prevention** → **Policies** and review what already exists — which

templates are on, in which mode, over which locations.

1. Run `DLPCUSTOM-Assess-DlpEnvironment.ps1 -CsvPath .\DlpDiscovery.csv`. Read-only, it captures: existing

DLP policies + rules and their **mode** (`Enable` / `TestWithNotifications` / `TestWithoutNotifications` / `Disable`) and locations via `Get-DlpCompliancePolicy` / `Get-DlpComplianceRule`; the available **sensitive information types** via `Get-DlpSensitiveInformationType`; and the DLP-relevant licensing. It writes `Custom DLP – Discovery – Assess`.

1. Note any **over-firing** existing policy and confirm the **SITs your design needs are present**

(Requirement 2.5). **Pass/fail:** you have an inventory of current DLP, the available SITs, and the licensed workloads — and you know which SITs/labels your new policy can reference.

Step 2 — Design conditions, actions, and exceptions (Design)

1. From the stakeholders' data-protection intent (Requirement 2.6), build the design matrix `Custom DLP – Sensitive Data – Design`. For each rule, specify:

• **Conditions** — which **SIT(s)** at which **confidence** (low/medium/high), **instance count** (min/max), any **sensitivity label**, and content/recipient conditions (e.g. *sent outside the organisation*).

• **Actions** — staged: **audit only** → **notify (policy tip)** → **restrict/block** → **encrypt** — and whether to **generate an incident report**.

• **Exceptions (`ExceptIf…`)** — the approved flows that must **not** trigger (approved partner domains, service mailboxes, specific sites/groups).

1. Choose the **workloads** (Exchange, SharePoint, OneDrive, Teams, endpoints) and the **pilot scope** (a pilot user group / site set, not all users).

1. **Pass/fail:** every rule maps to a documented intent; each condition references a SIT/label that exists (Step 1); the sensitive-action rows have named approvers. Determinations about *what is sensitive / which regulation applies* are the stakeholders' — recorded, not decided here.

Step 3 — Configure the policy in SIMULATION / test mode (Configure, simulate-first)

1. Dry-run first: `DLPCUSTOM-New-DlpPolicy.ps1 -PolicyName 'Custom DLP – Sensitive Data – Pilot' -DesignPath .\DlpDesign.json -WhatIf`.

With `-WhatIf` (the default) it makes **no changes** — it prints exactly the `New-DlpCompliancePolicy` / `New-DlpComplianceRule` it *would* run, the mode, the locations, the conditions, and the exceptions, and flags any referenced SIT/label that is missing in the tenant.

1. Read every line against `Custom DLP – Sensitive Data – Design`. Confirm the mode is a **test mode** (`TestWithoutNotifications` by default — evaluates content and records matches **without blocking or notifying**), the scope is the **pilot** scope, and the exceptions are present.

1. Apply in simulation: run with `-Commit`. The script creates the policy with

`New-DlpCompliancePolicy -Mode TestWithoutNotifications` scoped to the pilot locations, then adds the rule with `New-DlpComplianceRule` (conditions via `-ContentContainsSensitiveInformation`, exceptions via the `-ExceptIf…` parameters, actions per the design but with **no block** while in test).

1. **Pass/fail + rollback:** `Get-DlpCompliancePolicy -Identity 'Custom DLP – Sensitive Data – Pilot'`

shows the policy in a **test** mode over the pilot scope. Rollback is `Remove-DlpCompliancePolicy -Identity 'Custom DLP – Sensitive Data – Pilot'` (the script prints the exact reverse command). **Nothing is blocked or shown to users at this stage.**

Step 4 — Pilot: let the policy simulate (Pilot)

1. Leave the policy in test mode over the pilot scope so it evaluates real content and **records matches

with zero user impact**. Allow time for content to be processed (new/edited items evaluate as they change; existing content is evaluated as the service scans it — allow up to ~24–48 hours for meaningful volume, and longer for large SharePoint/OneDrive estates).

1. Do **not** move to enforcement yet. This simulation period is the mandatory high-impact-change gate for

the Purview category. **Pass/fail:** the policy is generating simulation matches over the pilot scope.

Step 5 — Validate and tune against the match evidence (Validate)

1. Run `DLPCUSTOM-Test-DlpPolicy.ps1 -PolicyName 'Custom DLP – Sensitive Data – Pilot'` — read-only, it

confirms the policy/rule mode, locations, conditions, and exceptions are exactly as designed, and writes `Custom DLP – Simulation – Validate`.

1. Run `DLPCUSTOM-Report-DlpMatches.ps1 -PolicyName 'Custom DLP – Sensitive Data – Pilot' -CsvPath .\DlpMatches.csv`

(read-only) to pull the simulation matches (from Activity Explorer / DLP reporting — verify the reporting surface in your tenant) into `Custom DLP – Matches – Report`.

1. **Tune** with `DLPCUSTOM-New-DlpPolicy.ps1 … -Update` (dry-run `-WhatIf`, then `-Commit`):

• **Too many false positives** → raise the **confidence** requirement, raise the **instance-count**

minimum, add an **exception** for the approved flow, or narrow the **SIT**.

• **False negatives (missed data)** → add/broaden a **SIT**, lower an over-strict threshold, or add a

content/recipient condition.

1. Re-simulate and re-report after each change. **Pass/fail:** match quality meets the agreed gate (e.g.

false-positive rate under the target, no known-sensitive sample missed) recorded in the Testing Guide.

Step 6 — Expand to enforcement, staged, under change control (Expand)

1. With the tuned policy passing the gate, move it toward enforcement **in stages** — dry-run `-WhatIf`

then `-Commit` each time:

• First to `TestWithNotifications` (simulation **plus** policy tips) so users start seeing guidance while nothing is yet blocked; or straight to `Enable` with only the **notify/policy-tip** action.

• Then, once notifications are validated, enable the **restrictive action** (restrict access / block / encrypt) per the design. The script sets mode via `Set-DlpCompliancePolicy -Mode Enable` and the action via `Set-DlpComplianceRule`.

1. Broaden **scope** from the pilot group to the full in-scope population only after the enforced policy is confirmed healthy on the pilot.

1. **Pass/fail + rollback:** the policy reports `Enable` with the intended action over the intended scope, and legitimate business flows still succeed (exceptions hold). Rollback at any stage is `Set-DlpCompliancePolicy -Mode TestWithoutNotifications` (back to no-impact simulation) or `-Mode Disable` — both printed by the script; keep the change window open until validated.

Order of operations

`Assess environment + SIT inventory → design conditions/actions/exceptions → dry-run (-WhatIf) policy → create in TestWithoutNotifications on pilot scope → simulate (no impact) → validate + report matches → tune (re-simulate) → stage to TestWithNotifications / Enable with tips → enable restrictive action → broaden scope — each move -WhatIf first, under change control, rollback ready.`

5. Testing Guide — Custom DLP Policy Development (Microsoft Purview)

Run after building the discovery baseline and after each configuration step. Every test is safe (read-only or a `-WhatIf` preview). Validate on the pilot scope in **simulation** before any move to enforcement.

Test 1 — Environment discovery is complete

Do: Run `DLPCUSTOM-Assess-DlpEnvironment.ps1`; compare its policy/SIT/licensing output against the Purview portal `Data loss prevention → Policies` and `Billing → Licenses`. **Expected:** Existing DLP policies, their modes and locations, the available sensitive info types, and the DLP-relevant licensing all match the portal; you can confirm the SITs your design needs are present.

Test 2 — Policy creation dry-run makes no changes

Do: Run `DLPCUSTOM-New-DlpPolicy.ps1 -PolicyName 'Custom DLP – Sensitive Data – Pilot' -DesignPath .\DlpDesign.json -WhatIf`; afterward run `Get-DlpCompliancePolicy` and confirm the policy does not exist yet. **Expected:** The dry-run prints the exact `New-DlpCompliancePolicy` / `New-DlpComplianceRule` it would run (mode, locations, conditions, exceptions) and the reverse (`Remove-DlpCompliancePolicy`) command; no policy was created — `-WhatIf` truly changed nothing. Any missing referenced SIT/label is flagged.

Test 3 — Policy is created in simulation / test mode, on the pilot scope

Do: After `-Commit`, run `DLPCUSTOM-Test-DlpPolicy.ps1 -PolicyName 'Custom DLP – Sensitive Data – Pilot'` (or `Get-DlpCompliancePolicy`/`Get-DlpComplianceRule`). **Expected:** The policy reports a **test** mode (`TestWithoutNotifications`), is scoped to the **pilot** locations only, its rule conditions and exceptions match `Custom DLP – Sensitive Data – Design`, and **no block action is active**. No user is notified and nothing is blocked.

Test 4 — Simulation generates matches with no user impact

Do: After the simulation period, run `DLPCUSTOM-Report-DlpMatches.ps1 -PolicyName 'Custom DLP – Sensitive Data – Pilot'`; have a pilot user perform a would-match action (e.g. email a test document containing the sample data). **Expected:** The match is **recorded** in `Custom DLP – Matches – Report`, but the pilot user sees **no policy tip and no block** (test-without-notifications) — DLP is observing, not enforcing.

Test 5 — Known-sensitive samples are caught (false-negative check)

Do: Seed the pilot scope with **known-sensitive test samples** for each data class the policy targets; re-run the match report. **Expected:** Every seeded sample appears as a match — the conditions detect the data they are meant to. Any miss is a false negative to tune (add/broaden the SIT, lower an over-strict threshold).

Test 6 — Approved flows are not caught (false-positive / exception check)

Do: Have a pilot user perform an **approved** flow that the exceptions cover (e.g. send to an approved partner domain, or from a service mailbox); review the match report. **Expected:** The approved flow does **not** register as a policy match — the `ExceptIf…` exceptions hold. An unexpected match is a false positive to tune (add/adjust the exception, raise confidence/instance count).

Test 7 — Tuning changes are previewed before applied

Do: Run a tuning change with `DLPCUSTOM-New-DlpPolicy.ps1 … -Update -WhatIf`, then `-Commit`; re-run the match report after re-simulation. **Expected:** The `-WhatIf` shows the exact `Set-DlpComplianceRule`/`Set-DlpCompliancePolicy` change and its reverse; after `-Commit` and re-simulation the false-positive/negative counts move in the intended direction.

Test 8 — Enforcement cut-over is staged and reversible

Do: Move the tuned policy to `TestWithNotifications` (or `Enable` with tips only) via `DLPCUSTOM-New-DlpPolicy.ps1 … -Update -Mode TestWithNotifications -Commit`; confirm with `DLPCUSTOM-Test-DlpPolicy.ps1`; then verify an approved flow still succeeds. **Expected:** Policy tips appear while nothing is blocked; approved flows still work; the script prints the rollback (`Set-DlpCompliancePolicy -Mode TestWithoutNotifications` / `-Mode Disable`). Only after this does the restrictive action get enabled.

Pass/fail gate

Gate	Requirement
Simulation pass	Tests 1–4 pass on the pilot scope: environment discovered, policy created in a test mode with no user impact, matches recorded
Tuning pass	Tests 5–7 pass: known-sensitive samples caught, approved flows excepted, false-positive/negative rates within the agreed target after tuning
Enforcement pass	Test 8 passes: staged cut-over (notify → restrict/block), approved flows still succeed, rollback confirmed available — before broadening scope beyond the pilot

6. Troubleshooting Guide — Custom DLP Policy Development (Microsoft Purview)

Field issues and how to resolve them, in the order you are likely to hit them. Every check here is safe.

DLP does not appear, or a workload can't be targeted

Symptom: `Data loss prevention` is missing from the Purview portal, or you can't scope a policy to endpoints. **Cause:** DLP is licensing-gated — baseline DLP (Exchange/SharePoint/OneDrive/Teams) is broadly licensed, but **Endpoint DLP** and advanced classification inputs (trainable classifiers, EDM) need **E5 / E5 Compliance**. **Fix:** Run `DLPCUSTOM-Assess-DlpEnvironment.ps1` and confirm the SKU in `Billing → Licenses`. Target only the licensed workloads; record the rest as deferred. Verify availability against **your own tenant and plan** — the script's mapping is indicative.

`Connect-IPPSSession` fails or DLP cmdlets are not found

Symptom: Connection errors, repeated auth prompts, or `Get-DlpCompliancePolicy` "is not recognized." **Cause:** Missing/old ExchangeOnlineManagement module, or basic-auth/legacy sign-in — DLP cmdlets live in **Security & Compliance PowerShell**, reached via `Connect-IPPSSession` (not `Connect-ExchangeOnline`). **Fix:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser -Force`; use a modern-auth/MFA admin account and connect with `Connect-IPPSSession`. Close/reopen PowerShell if a stale session is cached.

A referenced sensitive info type "does not exist"

Symptom: `DLPCUSTOM-New-DlpPolicy.ps1 -WhatIf` prints `[MISSING SIT]`, or a rule detects nothing. **Cause:** The condition references a SIT/label that isn't in the tenant — commonly a **custom** identifier (your customer-ID format) that was never built, or a name typo. **Fix:** Run `DLPCUSTOM-Assess-DlpEnvironment.ps1 -NeededSit '<name>'` to confirm presence and exact name. Build any missing **custom SIT / Exact Data Match / document fingerprint / trainable classifier** in its own dedicated solution first, then reference it here. Do **not** invent a SIT inside this policy work.

The policy is over-firing (too many false positives)

Symptom: The simulation match report is full of matches on content that isn't really sensitive. **Cause:** Conditions are too broad — low confidence, a low instance-count minimum, an overly generic SIT, or a missing exception for an approved flow. **Fix:** Tune with `DLPCUSTOM-New-DlpPolicy.ps1 … -Update` (dry-run `-WhatIf`, then `-Commit`): raise the **confidence** requirement, raise the **instance-count** minimum, narrow the **SIT**, or add an `ExceptIf…` **exception** for the approved flow. Re-simulate and re-report. Precision is the goal — an over-firing policy trains users to ignore DLP.

The policy misses known-sensitive data (false negatives)

Symptom: Seeded known-sensitive test samples do **not** appear in the match report. **Cause:** The condition is too strict (confidence/instance count set too high), the wrong SIT is used, or the sample sits in a workload the policy doesn't cover. **Fix:** Lower the over-strict threshold, add/broaden the **SIT**, or extend the policy's **locations**. Confirm the sample is in a targeted workload. Re-simulate; every seeded sample should match.

A pilot user sees a policy tip or a block during simulation

Symptom: Users are being notified or blocked while you intended a no-impact pilot. **Cause:** The policy is **not** in `TestWithoutNotifications` — it may be in `TestWithNotifications` (tips on) or already `Enable` (enforcing). **Fix:** Run `DLPCUSTOM-Test-DlpPolicy.ps1 -PolicyName '<name>'` to read the mode. To return to no-impact simulation: `Set-DlpCompliancePolicy -Identity '<name>' -Mode TestWithoutNotifications` (the author script prints this reverse command). Only move to notifications/enforcement deliberately, after the tuning gate.

Simulation shows no matches at all

Symptom: The match report is empty even though matching content exists. **Cause:** Content hasn't been evaluated yet (new/edited items evaluate as they change; existing content is scanned over time — allow ~24–48 hours, longer for large SharePoint/OneDrive estates), or the policy scope doesn't include where the content lives, or the reporting look-back window is too short. **Fix:** Wait and re-run `DLPCUSTOM-Report-DlpMatches.ps1 -Days <n>`; confirm the pilot scope covers the content's location with `DLPCUSTOM-Test-DlpPolicy.ps1`; seed a fresh test item to force near-term evaluation.

Matches can't be exported by the reporting script

Symptom: `DLPCUSTOM-Report-DlpMatches.ps1` reports the export was unavailable. **Cause:** DLP reporting surfaces evolve — several legacy `Get-Dlp*Report` cmdlets were retired in favour of **Activity Explorer**; `Export-ActivityExplorerData` may not be present/consented in your tenant. **Fix:** Review matches in the portal instead: **Data loss prevention** → **Alerts**, or **Data classification** → **Activity explorer** filtered to `Activity = DLPRuleMatch` and your policy (verify path). The script prints this fallback. The reporting *method* varies; the *evidence* (matches) is the same.

Endpoint DLP policy has no effect on devices

Symptom: A policy scoped to endpoints records nothing from devices. **Cause:** Devices aren't **onboarded to Purview** (separate from Intune/device management), or Endpoint DLP isn't licensed (E5 / E5 Compliance). **Fix:** Run `DLPCUSTOM-Scope-Health.ps1 -TargetEndpoint` to report readiness. Onboarding devices to Purview (`Settings → Device onboarding`, verify path) is performed in the

endpoint-DLP solution — this policy targets endpoints once they're onboarded. Record the gap as a hand-off item.

An approved business flow is being blocked after enforcement

Symptom: A legitimate flow (partner email, service mailbox, a specific site) is blocked once the policy is enforcing. **Cause:** The exception for that flow is missing or wrong — or you moved to a restrictive action before the exceptions were validated in simulation. **Fix:** Immediately roll back to no-impact simulation (`Set-DlpCompliancePolicy -Mode TestWithoutNotifications`) or `-Mode Disable`, add/correct the `ExceptIf…` exception with `DLPCUSTOM-New-DlpPolicy.ps1 … -Update`, re-simulate to confirm the flow no longer matches, then re-enforce. This is exactly why exceptions are validated (Testing Guide Test 6) **before** any block is enabled.

When to stop and escalate

• A required capability/workload is **not licensed** → scope the policy to what you have and record the rest

as deferred; do not attempt to force it.

• You are asked to decide **what data is sensitive, which regulation applies, or a records/retention

implication** → escalate to compliance, privacy, records, or legal. This solution provides the technical policy and the match evidence, never the determination.

• A portal path or reporting surface here **doesn't match your tenant** → the Purview DLP portal and

reporting surfaces are actively evolving; verify the current path/method in your tenant before assuming the guide is wrong about the *step*.

7. Best Practices — Custom DLP Policy Development (Microsoft Purview)

• **Never create a DLP policy directly in enforcement.** Always author in `TestWithoutNotifications`

first — it evaluates content and records matches with **zero user impact**. Enforcement is something you *promote a tuned policy to*, never the starting state. This is the whole reason the Purview model pilots in simulation.

• **Tune before you enforce, and tune against evidence.** Use the simulation match report to drive down

false positives (raise confidence, raise instance-count minimums, add exceptions, narrow the SIT) and false negatives (add/broaden SITs, lower over-strict thresholds). A block action on an untuned policy is how DLP breaks legitimate work on day one.

• **False positives are a security problem.** Every over-firing rule trains users to dismiss DLP warnings,

so the warning that matters gets dismissed too. Precision *is* the security control — not an aesthetic.

• **Encode approved flows as explicit exceptions.** The business has legitimate sharing paths (partner

domains, service mailboxes, specific sites). Capture them as `ExceptIf…` exceptions so the policy protects data without becoming the thing everyone routes around.

- **Prefer high confidence and instance counts over blunt matches.** A single SSN-shaped string is noisy;

ten at *high* confidence going *outside the organisation* is a real signal. Design conditions for signal, not for maximum firing.

- **Consume data classes; don't invent them here.** If a condition needs a data class Purview lacks, build

the **custom SIT / Exact Data Match / document fingerprint / trainable classifier** in its own solution first, then reference it. Don't stretch a built-in SIT to fit — accuracy comes from the right classifier.

- **Stage the actions, not just the mode.** Move audit-only → policy tip/notify → restrict/block →

encrypt. Let users learn from tips before anything is blocked; validate each stage before the next.

- **Always dry-run policy changes.** Run every create/update with `-WhatIf` and read it against the design

before `-Commit`; the scripts print the exact reverse command so rollback to no-impact simulation (`-Mode TestWithoutNotifications`) or `Disable` is one line.

- **Pilot on a scope, then expand.** Simulate and tune on a pilot user/site set; broaden to the full

population only after the enforced policy is confirmed healthy on the pilot and approved flows still work.

- **Treat portal paths and the simulation UI as provisional.** The Purview DLP portal and simulation

experience evolve — verify paths in your tenant rather than trusting last quarter's screenshot.

- **Determinations belong to compliance/privacy/legal.** *What* is sensitive, *which* regulation applies,

and any records/retention implication are theirs to decide — you provide the technical policy and the match evidence, not the ruling.

8. Deliverables Summary — Custom DLP Policy Development (Microsoft Purview)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	How custom DLP fits your data — conditions (SITs, confidence, instance count, labels, recipients), actions, exceptions, the simulate-first tuning loop, limitations, and the naming convention

Document	Purpose
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing (DLP licensing, least-privilege role, S&C PowerShell, workloads, the SITs/labels the conditions need, data-protection intent, pilot + change control)
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered portal + S&C PowerShell steps: assess, design conditions/actions/exceptions, create the policy in simulation, pilot, validate & tune against matches, expand to enforcement — each policy/rule named, `-WhatIf` first, pass/fail + rollback
5. Testing Guide	Validation procedures with simulation / tuning / enforcement pass-fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`DLPCUSTOM-Assess-DlpEnvironment.ps1`	**Discovery** (read-only): inventories existing DLP policies + rules and their **mode**/locations (`Get-DlpCompliancePolicy`/`Get-DlpComplianceRule`), lists available **sensitive info types** (`Get-DlpSensitiveInformationType`), and reports DLP-relevant licensing. Writes `Custom DLP – Discovery – Assess`
`DLPCUSTOM-New-DlpPolicy.ps1`	**Configuration / remediation** (`-WhatIf` by default, `-Commit` to apply): creates a custom DLP policy **in simulation** (`New-DlpCompliancePolicy -Mode TestWithoutNotifications`) with its rule (conditions, exceptions, actions) from a design file; `-Update` tunes an existing policy/rule and `-Mode` stages it toward enforcement. Prints the exact reverse command for every change
`DLPCUSTOM-Test-DlpPolicy.ps1`	**Validation** (read-only): confirms a policy/rule's **mode**, locations, conditions, and exceptions match the design and that it is in a test mode before enforcement. Writes `Custom DLP – Simulation – Validate`
`DLPCUSTOM-Report-DlpMatches.ps1`	**Reporting** (read-only): pulls simulation/enforcement **matches** for a policy from the DLP reporting / Activity Explorer surface into a CSV for false-positive/negative tuning. Writes `Custom DLP – Matches – Report`

Script	What it does
`DLPCUSTOM-Scope-Health.ps1`	**Scope / health helper** (read-only): confirms the in-scope M365 workloads, reports endpoint device-onboarding readiness, and checks that the SITs/labels the design references exist. Writes `Custom DLP – Scope – Map`

All scripts are **safe by default** — read-only, or a `-WhatIf` preview requiring an explicit `-Commit` to change anything, and every policy is created **in simulation/test mode**, never straight to enforcement. No credentials or secrets are stored in code; every script uses interactive modern-auth (`Connect-IPPSSession`).

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the model: **Assess → Design → Configure → Pilot (simulate) → Validate →

Expand (enforce).**

Outcome

Custom DLP policies tuned to your exact data in Microsoft Purview — tailored conditions, sensitive info types, exceptions, and staged actions — piloted in **simulation** with no user impact, tuned against real match evidence, and promoted to enforcement under change control, so the protection is in place, credible, and defensible. Determinations about what is sensitive and which regulations apply are deferred to compliance, privacy, and legal.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

DLPCUSTOM-Assess-DlpEnvironment.ps1

Purpose: Inventories the current Microsoft Purview DLP environment before you author a custom policy: every DLP policy and rule with its MODE (Enable / TestWithNotifications / TestWithoutNotifications / Disable) and locations, the available sensitive information types, and the DLP-relevant licensing. READ-ONLY — makes no changes. Writes an evidence CSV: 'Custom DLP - Discovery - Assess'. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DLPCUSTOM-Assess-DlpEnvironment.ps1 -CsvPath .\DlpDiscovery.csv Optional: -NeededSit 'Credit Card Number','Custom Customer ID' (flags whether each exists in the tenant) PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession for Security & Compliance PowerShell). EXPECTED RESULTS PASS = you have an inventory of existing DLP policies/modes/locations, the available SITs, and the licensed workloads, and every SIT your design needs is confirmed present. No object is modified. .NOTES Purview — Custom DLP Policy Development. Read-only assessment; no secrets in code (interactive modern-auth). Capability availability is licensing-dependent — verify against your own tenant and plan. > [CmdletBinding()] param([string]$CsvPath = ".\Custom-DLP-Discovery-Assess.csv", [string[]]$NeededSit)

How to run: powershell -ExecutionPolicy Bypass -File .\DLPCUSTOM-Assess-DlpEnvironment.ps1 -CsvPath .\DlpDiscovery.csv Optional: -NeededSit 'Credit Card Number','Custom Customer ID' (flags whether each exists in the tenant) PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession for Security & Compliance PowerShell). EXPECTED RESULTS PASS = you have an inventory of existing DLP policies/modes/locations, the available SITs, and the licensed workloads, and every SIT your design needs is confirmed present. No object is modified. .NOTES Purview — Custom DLP Policy Development. Read-only assessment; no secrets in code (interactive modern-auth). Capability availability is licensing-dependent — verify against your own tenant and plan. > [CmdletBinding()] param([string]$CsvPath = ".\Custom-DLP-Discovery-Assess.csv", [string[]]$NeededSit)

DLPCUSTOM-New-DlpPolicy.ps1

Purpose: Authors a custom Microsoft Purview DLP policy and its rule from a JSON design file: conditions (sensitive info types + confidence + instance counts), exceptions (approved flows), and staged actions. SAFE BY DEFAULT — previews with -WhatIf and changes nothing until -Commit, and the policy is created in SIMULATION (New-DlpCompliancePolicy -Mode TestWithoutNotifications), never straight to enforcement. Use -Update to tune an existing policy/rule, and -Mode to stage it toward enforcement. Prints the exact reverse (rollback) command for every change. HOW TO RUN Dry-run create (no changes): powershell -ExecutionPolicy Bypass -File .\DLPCUSTOM-New-DlpPolicy.ps1 -PolicyName 'Custom DLP - Sensitive Data - Pilot' -DesignPath .\DlpDesign.json -WhatIf Create in simulation: powershell -ExecutionPolicy Bypass -File .\DLPCUSTOM-New-DlpPolicy.ps1 -PolicyName 'Custom DLP - Sensitive Data - Pilot' -DesignPath .\DlpDesign.json -Commit Tune an existing policy's rule, then re-simulate: powershell ... -PolicyName 'Custom DLP - Sensitive Data - Pilot' -DesignPath .\DlpDesign.json -Update -Commit Stage toward enforcement (only after tuning passes the gate): powershell ... -PolicyName 'Custom DLP - Sensitive Data - Pilot' -Update -Mode TestWithNotifications -Commit DesignPath JSON (example): { "RuleName":"Custom DLP Rule - Sensitive Data - R1", "ExchangeLocation":["All"], "SharePointLocation":["All"], "OneDriveLocation":["All"], "PilotScope":{"ExchangeSenderMemberOf":["DLP-Pilot@contoso.com"]}, "Sit":[{"Name":"Credit Card

Number","MinCount":10,"Confidence":85}],
"Actions":{"NotifyUser":["Owner"],"BlockAccess":false,"GenerateIncidentReport":["Owner"]},
"ExceptIfRecipientDomainIs":["approved-partner.example"] } PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession). EXPECTED RESULTS Dry-run prints the exact New-/Set-DlpCompliancePolicy + New-/Set-DlpComplianceRule it would run, the MODE, locations, conditions and exceptions, and each reverse command; flags any missing referenced SIT. With -Commit (create), Get-DlpCompliancePolicy shows the policy in TestWithoutNotifications on the pilot scope with NO block active. PASS = policy exists in simulation, matching the design. .NOTES Purview — Custom DLP Policy Development. NEVER creates a policy in Enable mode; enforcement is reached only via -Update -Mode after the simulation match evidence passes the gate. Determinations about what is sensitive / which regulation applies are deferred to compliance/privacy/legal. No secrets in code. > [CmdletBinding(DefaultParameterSetName='WhatIf')] param([Parameter(Mandatory)][string]$PolicyName, [string]$DesignPath, [switch]$Update, [ValidateSet('TestWithoutNotifications','TestWithNotifications','Enable','Disable')] [string]$Mode = 'TestWithoutNotifications', [switch]$Commit, [switch]$WhatIf = $true) if ($Commit) { $WhatIf = $false }

How to run: Dry-run create (no changes): powershell -ExecutionPolicy Bypass -File .\DLPCUSTOM-New-DlpPolicy.ps1 -PolicyName 'Custom DLP - Sensitive Data - Pilot' -DesignPath .\DlpDesign.json -WhatIf Create in simulation: powershell -ExecutionPolicy Bypass -File .\DLPCUSTOM-New-DlpPolicy.ps1 -PolicyName 'Custom DLP - Sensitive Data - Pilot' -DesignPath .\DlpDesign.json -Commit Tune an existing policy's rule, then re-simulate: powershell ... -PolicyName 'Custom DLP - Sensitive Data - Pilot' -DesignPath .\DlpDesign.json -Update -Commit Stage toward enforcement (only after tuning passes the gate): powershell ... -PolicyName 'Custom DLP - Sensitive Data - Pilot' -Update -Mode TestWithNotifications -Commit DesignPath JSON (example): { "RuleName":"Custom DLP Rule - Sensitive Data - R1", "ExchangeLocation":["All"], "SharePointLocation":["All"], "OneDriveLocation":["All"], "PilotScope":{"ExchangeSenderMemberOf":["DLP-Pilot@contoso.com"]}, "Sit":[{"Name":"Credit Card Number","MinCount":10,"Confidence":85}],
"Actions":{"NotifyUser":["Owner"],"BlockAccess":false,"GenerateIncidentReport":["Owner"]},
"ExceptIfRecipientDomainIs":["approved-partner.example"] } PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession). EXPECTED RESULTS Dry-run prints the exact New-/Set-DlpCompliancePolicy + New-/Set-DlpComplianceRule it would run, the MODE, locations, conditions and exceptions, and each reverse command; flags any missing referenced SIT. With -Commit (create), Get-DlpCompliancePolicy shows the policy in TestWithoutNotifications on the pilot scope with NO block active. PASS = policy exists in simulation, matching the design. .NOTES Purview — Custom DLP Policy Development. NEVER creates a policy in Enable mode; enforcement is reached only via -Update -Mode after the simulation match evidence passes the gate. Determinations about what is sensitive / which regulation applies are deferred to compliance/privacy/legal. No secrets in code. > [CmdletBinding(DefaultParameterSetName='WhatIf')] param([Parameter(Mandatory)][string]$PolicyName, [string]$DesignPath, [switch]$Update, [ValidateSet('TestWithoutNotifications','TestWithNotifications','Enable','Disable')] [string]$Mode = 'TestWithoutNotifications', [switch]$Commit, [switch]$WhatIf = $true) if ($Commit) { $WhatIf = $false }

DLPCUSTOM-Report-DlpMatches.ps1

Purpose: Pulls DLP match/detection events for a custom policy from the Microsoft Purview reporting surface (Activity Explorer via Export-ActivityExplorerData) so you can review simulation results and tune: drive down false positives (add exceptions, raise confidence/instance counts) and false negatives

(broaden/add SITs). READ-ONLY — makes no changes. Writes 'Custom DLP - Matches - Report'. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DLPCUSTOM-Report-DlpMatches.ps1 -PolicyName 'Custom DLP - Sensitive Data - Pilot' -CsvPath .\DlpMatches.csv Optional: -Days 7 (look-back window; default 7) PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession). EXPECTED RESULTS PASS = a dated CSV of matches for the policy that lets you compute the false-positive rate and confirm known-sensitive samples were caught. If the reporting surface differs in your tenant, the script says so and points to the portal DLP reports / Activity Explorer. No object is modified. .NOTES Purview — Custom DLP Policy Development. Read-only. DLP reporting cmdlets/surfaces evolve — verify the available surface in your tenant (some legacy Get-Dlp*Report cmdlets are retired in favour of Activity Explorer). No secrets in code (interactive modern-auth). > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [string]$CsvPath = ".\Custom-DLP-Matches-Report.csv", [int]$Days = 7)

How to run: powershell -ExecutionPolicy Bypass -File .\DLPCUSTOM-Report-DlpMatches.ps1 -PolicyName 'Custom DLP - Sensitive Data - Pilot' -CsvPath .\DlpMatches.csv Optional: -Days 7 (look-back window; default 7) PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession). EXPECTED RESULTS PASS = a dated CSV of matches for the policy that lets you compute the false-positive rate and confirm known-sensitive samples were caught. If the reporting surface differs in your tenant, the script says so and points to the portal DLP reports / Activity Explorer. No object is modified. .NOTES Purview — Custom DLP Policy Development. Read-only. DLP reporting cmdlets/surfaces evolve — verify the available surface in your tenant (some legacy Get-Dlp*Report cmdlets are retired in favour of Activity Explorer). No secrets in code (interactive modern-auth). > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [string]$CsvPath = ".\Custom-DLP-Matches-Report.csv", [int]$Days = 7)

DLPCUSTOM-Scope-Health.ps1

Purpose: Pre-flight and health check for a custom DLP policy's scope: confirms the in-scope M365 workloads, reports whether devices are onboarded to Purview (needed for Endpoint DLP), and verifies the sensitive info types / sensitivity labels the design references actually exist in the tenant. READ-ONLY — makes no changes. Writes 'Custom DLP - Scope - Map'. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DLPCUSTOM-Scope-Health.ps1 -CsvPath .\DlpScope.csv -NeededSit 'Credit Card Number' -TargetEndpoint -TargetEndpoint : also check endpoint device-onboarding readiness (Endpoint DLP needs E5/E5 Compliance + onboarded devices) PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession). Optional: Microsoft.Graph for device readiness. EXPECTED RESULTS PASS = every targeted workload is confirmed present, endpoint readiness is reported accurately (if -TargetEndpoint), and each referenced SIT/label is PRESENT. Any gap is recorded as a hand-off item (e.g. endpoints not yet onboarded -> endpoint-DLP solution). No object is modified. .NOTES Purview — Custom DLP Policy Development. Read-only; no secrets in code (interactive modern-auth). Endpoint onboarding and cloud-app connection are separate solutions — this only reports readiness. > [CmdletBinding()] param([string]$CsvPath = ".\Custom-DLP-Scope-Map.csv", [string[]]$NeededSit, [string[]]$NeededLabel, [switch]$TargetEndpoint)

How to run: powershell -ExecutionPolicy Bypass -File .\DLPCUSTOM-Scope-Health.ps1 -CsvPath .\DlpScope.csv -NeededSit 'Credit Card Number' -TargetEndpoint -TargetEndpoint : also check endpoint device-onboarding readiness (Endpoint DLP needs E5/E5 Compliance + onboarded devices) PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession). Optional: Microsoft.Graph for device readiness. EXPECTED RESULTS PASS = every targeted workload is confirmed present, endpoint readiness is reported accurately (if -TargetEndpoint), and each referenced SIT/label is PRESENT. Any

gap is recorded as a hand-off item (e.g. endpoints not yet onboarded -> endpoint-DLP solution). No object is modified. .NOTES Purview — Custom DLP Policy Development. Read-only; no secrets in code (interactive modern-auth). Endpoint onboarding and cloud-app connection are separate solutions — this only reports readiness. > [CmdletBinding()] param([string]$CsvPath = ".\Custom-DLP-Scope-Map.csv", [string[]]$NeededSit, [string[]]$NeededLabel, [switch]$TargetEndpoint)

DLPCUSTOM-Test-DlpPolicy.ps1

Purpose: Validates a custom DLP policy against its design: confirms the policy MODE, the locations, and each rule's conditions (sensitive info types, confidence, instance counts), exceptions, and actions. Flags if a policy is in an ENFORCEMENT mode (Enable) so you can confirm that was intentional and gate-passed. READ-ONLY — makes no changes. Writes 'Custom DLP - Simulation - Validate'. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DLPCUSTOM-Test-DlpPolicy.ps1 -PolicyName 'Custom DLP - Sensitive Data - Pilot' Optional: -CsvPath .\DlpValidate.csv PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession). EXPECTED RESULTS PASS = the policy is in the expected mode (TestWithoutNotifications while piloting), scoped to the pilot locations, and its rule conditions/exceptions/actions match the design. A policy already in Enable mode is reported so the gate can be re-confirmed. No object is modified. .NOTES Purview — Custom DLP Policy Development. Read-only; no secrets in code (interactive modern-auth). > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [string]$CsvPath = ".\Custom-DLP-Simulation-Validate.csv")

How to run: powershell -ExecutionPolicy Bypass -File .\DLPCUSTOM-Test-DlpPolicy.ps1 -PolicyName 'Custom DLP - Sensitive Data - Pilot' Optional: -CsvPath .\DlpValidate.csv PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession). EXPECTED RESULTS PASS = the policy is in the expected mode (TestWithoutNotifications while piloting), scoped to the pilot locations, and its rule conditions/exceptions/actions match the design. A policy already in Enable mode is reported so the gate can be re-confirmed. No object is modified. .NOTES Purview — Custom DLP Policy Development. Read-only; no secrets in code (interactive modern-auth). > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [string]$CsvPath = ".\Custom-DLP-Simulation-Validate.csv")